

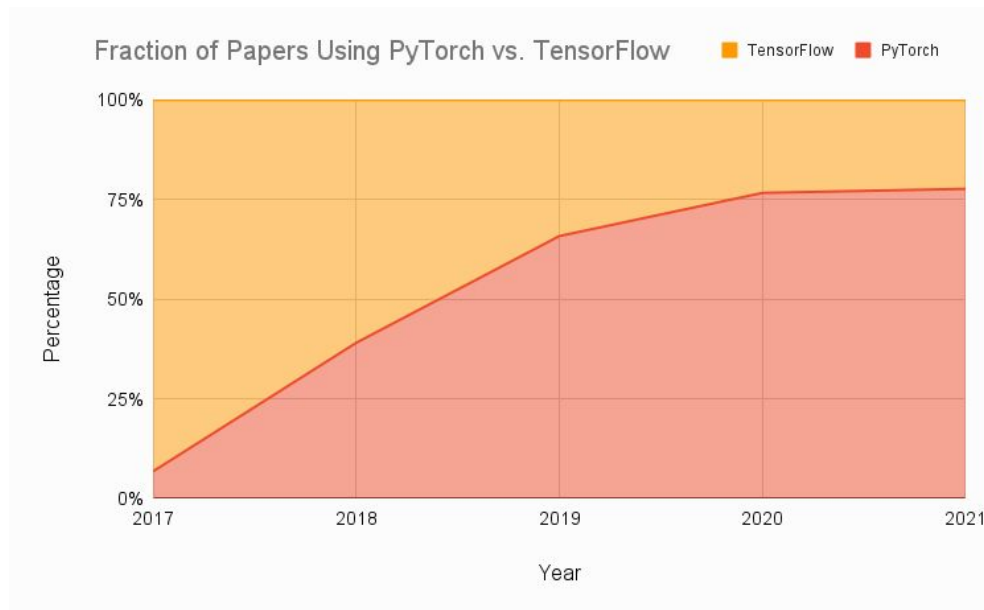
Carnegie Mellon University

PyTorch Tutorial & Project Topics

18794 Recitation 1

Deep Learning Framework

A deep learning framework is a library to design and train deep learning models.



Source:
AssemblyAI

Installation

<https://pytorch.org/get-started/locally/>

PyTorch Build	Stable (2.8.0)			Preview (Nightly)	
Your OS	Linux	Mac		Windows	
Package	Pip	LibTorch		Source	
Language	Python			C++ / Java	
Compute Platform	CUDA 12.6	CUDA 12.8	CUDA 12.9	ROCm 6.4	CPU
Run this Command:	<pre>pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126</pre>				

Tensors

Tensors are core data structure in PyTorch, it is a **multi-dimensional matrix** containing elements of a **single data type**, similar to NumPy's ndarrays.

Optimized and can be moved to various devices (**CUDA GPU**, CPU, MPS...)

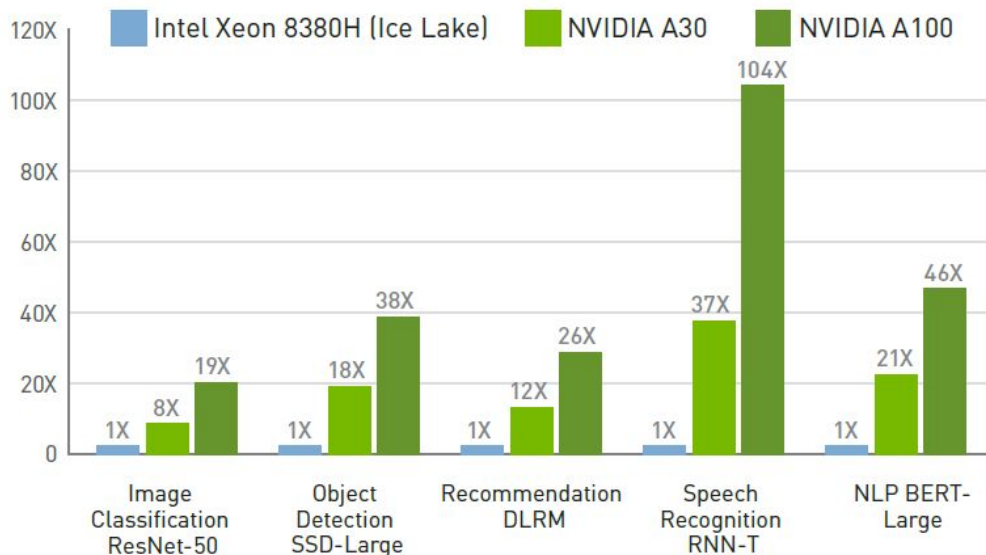
```
x = x.to("cuda")  # or "cpu", "mps"
```

Tensors are optimized for automatic differentiation.

Why GPU?

MLPerf 1.1 Inference Results

Server Scenario Per Processor Performance



CPU is a small group of experts,

GPU is a large group of primary school students (thousands of threads)

Tensor Initialization

from python sequences

```
torch.tensor([[1., -1.], [1., -1.]])
```

explicitly assigning data type and/or allocation device

```
torch.tensor([[1., -1.], [1., -1.]], dtype=torch.float32, device="cuda")
```

zero-copy construction from existing NumPy ndarrays

```
a = numpy.array([1, 2, 3])
```

```
t = torch.from_numpy(a)
```

Using functions similar to NumPy functions

```
torch.zeros([2, 4], dtype=torch.int32)
```

```
torch.ones([2, 4], dtype=torch.float64, device="cuda")
```

Images as Tensors

An image can be represented with tensor of shape (Channel, Height, Width)

- this is called “channel-first order”

Images are usually expressed as an RGB three **channel** array

A batch of images is (N, Channel, Height, Width)

Image Transformations

PyTorch Torchvision supports various computer vision transformations with **torchvision.transforms** package. **Most** transforms can be applied on both PIL image and torch tensor.

Example:

```
transforms.Compose()  # sequentially combine several transformations
transforms.ToTensor() # convert PIL Image to tensor
transforms.Normalize() # standardize values of each image based on mean/std
transforms.Resize()   # resize image
```

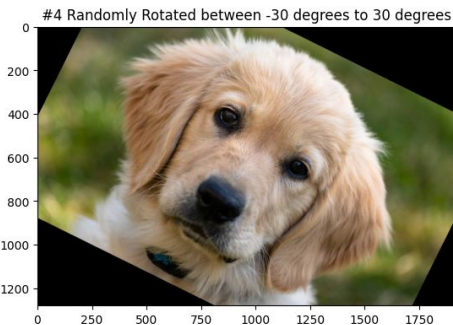
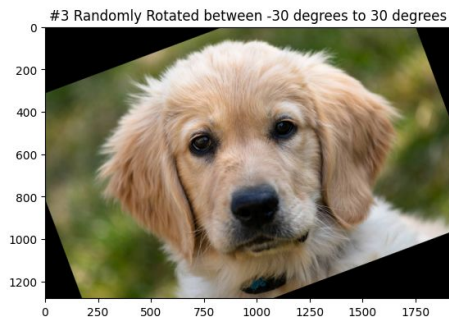
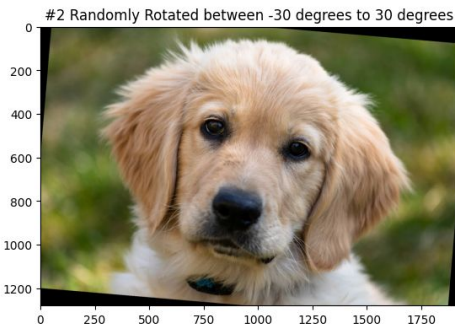
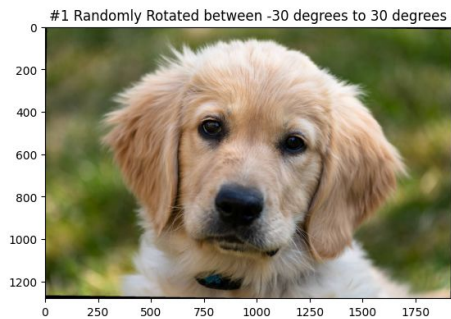
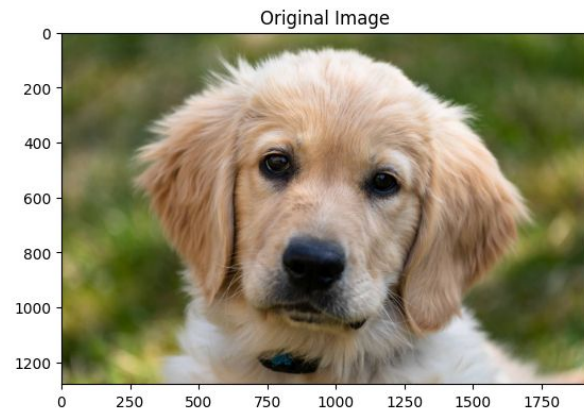
...

<https://docs.pytorch.org/vision/0.23/transforms.html>

Data Transformation

`transforms.RandomRotation`

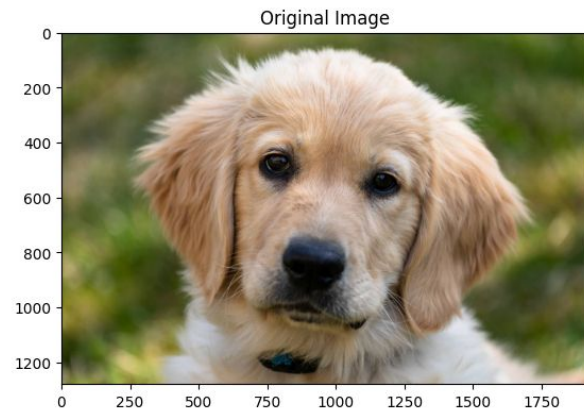
```
image = Image.open('puppy.jpeg')  
transform = transforms.RandomRotation((-30,30))
```



Data Transformation

`transforms.ColorJitter`

```
transform = transforms.ColorJitter(hue=0.5)
```



Why Transforming Images?

Deep Learning relies on large amounts of data

Transforming existing data supplements training to prevent overfitting

- overfitting is when model performs well on training data but not on testing data
- “Data Augmentation”

Creating Datasets

`torch.utils.data.Dataset` (https://pytorch.org/tutorials/beginner/basics/data_tutorial.html)

Provides a way to **store** and **access samples** and their **corresponding labels**.

Use torchvision predefined(<https://pytorch.org/vision/stable/datasets.html>) or custom datasets.

```
class CustomDataset(Dataset):
    def __init__(self, images, labels):
        self.images, self.labels = ... # Initialize the dataset

    def __len__(self):
        return len(self.images) # Return the length of the dataset

    def __getitem__(self, index): # get one sample from dataset
        # Return a/multiple tensor(s), apply necessary transformations
        return self.images[index], self.labels[index]
```

Creating Datasets

Three most important functions to implement for custom dataset:

- `__init__`
 - initialize the dataset, store information about image paths, labels, transforms, etc.
- `__len__`
 - tells how many samples are in the dataset
- `__getitem__`
 - get one sample from the dataset
 - *one sample can include model input, expected output (ground truth/label), and any other supplementary information*

Loading from Datasets

`torch.utils.data.DataLoader`

This module helps sampling, batching data from dataset

```
loader = DataLoader(dataset, batch_size=N, shuffle=False, sampler=None,  
                    batch_sampler=None, num_workers=0, collate_fn=None,  
                    pin_memory=False, drop_last=False, timeout=0,  
                    worker_init_fn=None, *, prefetch_factor=2,  
                    persistent_workers=False)
```

```
for images, labels in loader: # a batch N samples  
    ...
```

Loading from Datasets

Automatic batching

For example, in a usual image classification task,
we want **dataset.__getitem__** to return a tensor of shape (3, 128, 128) and a python integer.

Let batch size be 32, each time we collect a sample from the **DataLoader** iterator, we will get a tuple of a tensor of shape (32, 3, 128, 128) and a tensor of shape (32,)

Model Training

MLP Forward Propagation

X is the input to layer 1 neurons

$$Z^1 = X.W^1 + b^1 = (W^1)^T * X + b^1$$

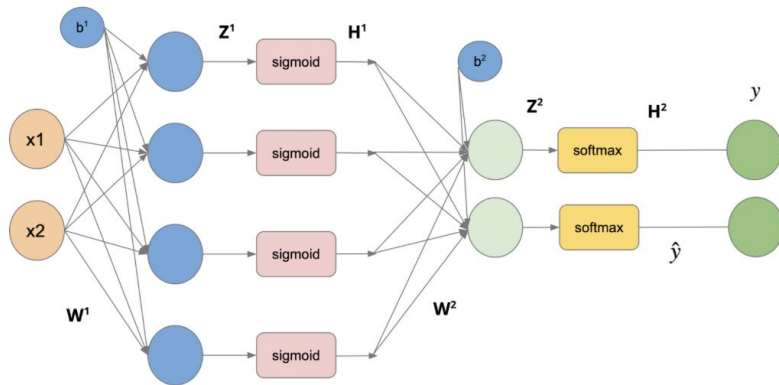
$$H^1 = \text{sigmoid}(Z^1)$$

H^1 is the input to output layer neurons.

$$Z^2 = H^1.W^2 + b^2 = (W^2)^T * H^1 + b^2$$

$$\hat{y} = \text{softmax}(Z_2)$$

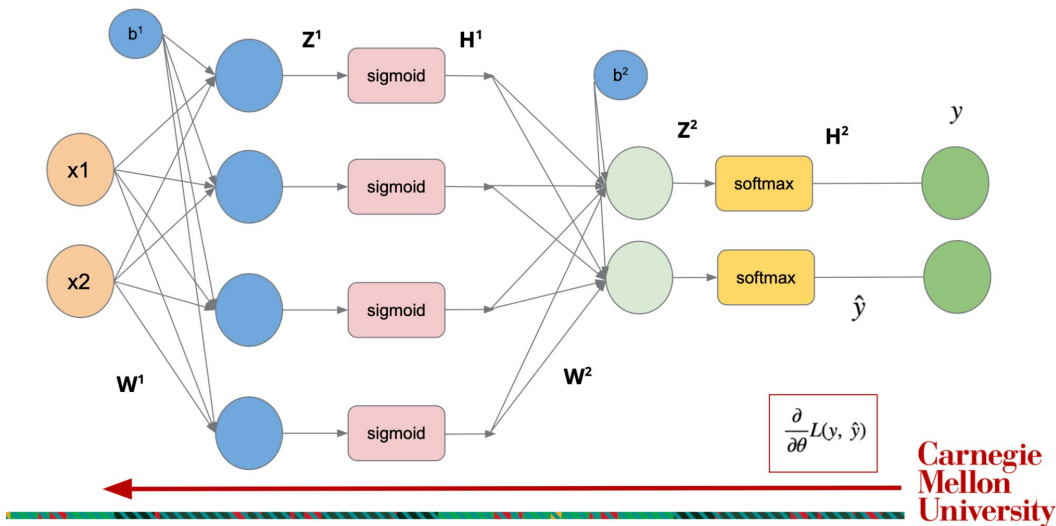
$\hat{y}$ is the output predicted by the model.



Model Training

As part of backpropagation, we need to compute multiple partial derivatives, this involves lot of computation.

MLP Backpropagation



Autograd

Autograd classes:

- backward
- grad

backward() function is used to compute the gradient of scalar value(loss function) with respect to all graph leaves(model parameters) involved.

grad property computes and returns the gradients of outputs with respect to specified Inputs.

```
x = torch.tensor([[1., -1.], [1., 1.]], requires_grad=True)
x
✓ 0.0s

tensor([[ 1., -1.],
        [ 1.,  1.]], requires_grad=True)
```

Scalar value

Graph leaf or parameter with require grad on

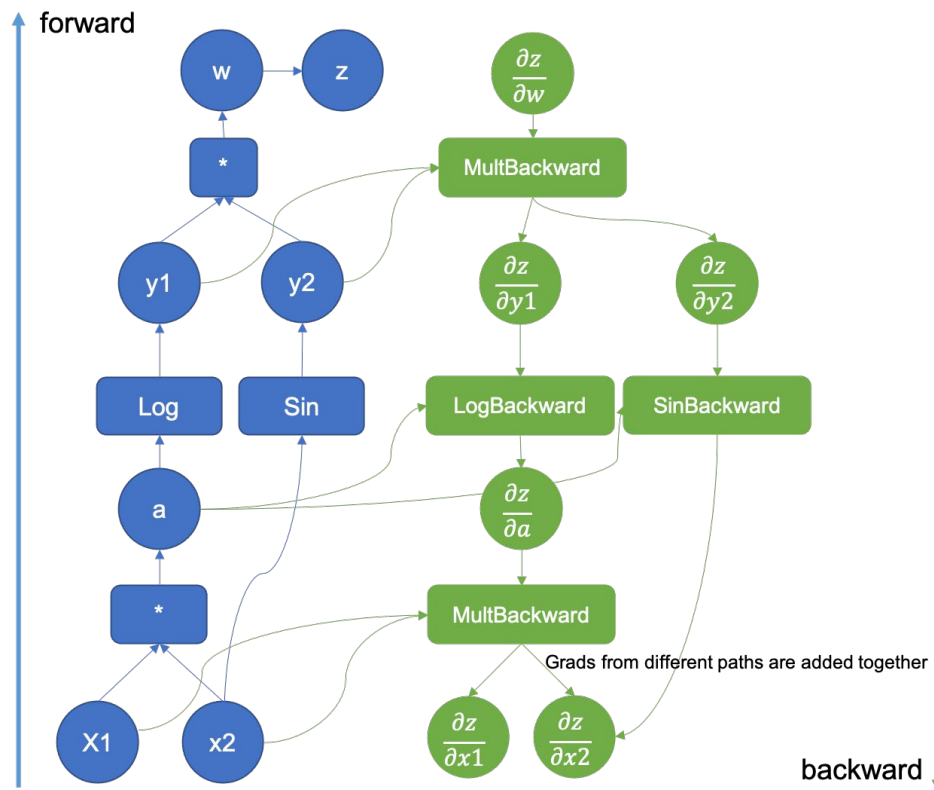
```
out = x.pow(2).sum()
out
✓ 0.0s

tensor(4., grad_fn=<SumBackward0>)

out.backward()
x.grad
✓ 0.1s

tensor([[ 2., -2.],
        [ 2.,  2.]])
```

Computational Graphs



Creating Neural Networks

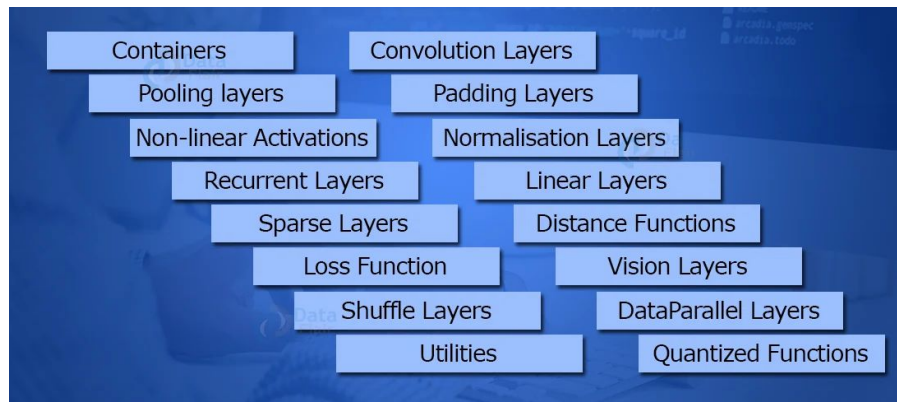
torch.nn (<https://pytorch.org/docs/stable/nn.html>)

Basic components of a model.

nn.Module class is the base class for all the neural network components.

Use **torch.nn** package to construct building blocks of neural network like

- Modules and Layers
- Loss Functions
- Activation Functions
- Containers
- Utility Functions
- etc



Creating Neural Networks

You can define your own (nested) model/layer/module... by subclassing **nn.Module**.

```
class CustomModel(nn.Module):  
    def __init__(self): # define all the layers and parameters here  
        super().__init__()  
        self.fc1 = nn.Linear(10, 20) # predefined linear layer  $y = wx + b$   
        self.fc2 = nn.Linear(20, 1)  
        self.activation = nn.ReLU()  
  
    def forward(self, x): # define the forward pass  
        x = self.activation(self.fc1(x))  
        x = self.fc2(x)  
        return x
```

Creating Neural Networks

`torchvision.models` (<https://pytorch.org/vision/stable/models.html>)

Contains definitions of various models for addressing different computer vision tasks, and includes model weights trained on large datasets.

```
from torchvision.models import resnet50, ResNet50_Weights
```

```
weights = ResNet50_Weights.DEFAULT
```

```
model = resnet50(weights=weights)
```

Train and Eval Modes

model.train()

model.eval()

Setting the modes impacts certain layers like DropOut and Batch Normalization.

DropOut: Some of the elements in input tensor are randomly dropped out with probability p during training. But while testing DropOut is not applied.

Batch Normalization: During training, Batch normalization uses the mean and variance of current batch to normalize the values but during testing it uses mean and variance that were accumulated during testing

Optimizers

`torch.optim` (<https://pytorch.org/docs/stable/optim.html>)

A PyTorch package for optimization algorithms, uses gradient calculated by `.backward()`/`.grad` to update model parameters with variants of gradient descent.

These are commonly used optimization algorithms from PyTorch:

- SGD
- Adam
- AdamW
- Adagrad
- RMSProp
- ...

```
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
```

Training Model

Basic training loop (<https://pytorch.org/tutorials/beginner/introyt/trainingyt.html>)

```
# One epoch (i.e. going over the whole dataset once)
model.train()
for images, labels in train_loader:
    optimizer.zero_grad() # clear the gradient from previous iteration
    outputs = model(images) # forward pass of the model
    loss = loss_fn(outputs, labels) # calculating loss
    loss.backward() # back propagation
    optimizer.step() # apply gradient descent
```

There are more: moving to device, scheduler, multiple models, etc.

Evaluating Model

Basic validation loop

```
model.eval()  
with torch.no_grad(): # context manager to disable gradient computation  
    for images, labels in test_loader:  
        outputs = model(images)  
        loss = loss_fn(outputs, labels)  
        # notice we don't call loss.backward() or optimizer.step()
```

There are more: moving to device, additional metrics, logging, etc.

Structure of Training Function

```
for epoch in range(epochs):  
    # training loop  
    for images, labels in train_loader:  
        ...  
  
    # validation loop  
    with torch.no_grad(): # context manager to disable gradient computation  
        for images, labels in test_loader:  
            ...  
  
    # logging  
    print(...)
```

Model Serialization

Model state dict contains all the model parameters and persistent buffers of the model.

```
# save model weights to file
```

```
torch.save(model.state_dict(), "model.pth")
```

```
# load model weights from saved file
```

```
model.load_state_dict(torch.load("model.pth"))
```

Final Project

Topic

Anything related to **Computer Vision** using **Deep Learning**.

You may train existing models on your own datasets, apply model to something novel.
It **cannot** be simply predicting with some off-the shelf models.

The project can be technical or practical.

- Detection
- Classification
- Segmentation
- Tracking
- Pose estimation
- Mixture of several tasks
- ...

Some Example Projects from past semesters

- Detecting and Predicting Produce Shelf Life
- 2D Person Detection, Tracking, and Global Geolocalization
- 3D Object Detection with RGB + Point Cloud
- Dynamic Gaussian Splatting + Novel View Synthesis
- ...

You will form project groups of **five**.

If you have special reasons to be in groups of four/six, send a request to TAs.

Projects will be in 3 stages:

- Proposal
- Mid-term Report
- **Final Report and Presentation**

Reports and Presentations

Reports: in a conference paper style, contains introduction, data, methods, experiments, conclusion, references... We will provide templates in LaTeX (learn LaTeX basics if you haven't!).

Presentations: the last lecture will be poster presentations for each group. You will present your topics methods, outcomes... with poster and **emos (!!! preferably live demo !!!)**.